COP509

Natural Language Processing

Retrieval-Augmented Generation (RAG)

Prof. Georgina Cosma

Department of Computer Science
Loughborough University

g.cosma@lboro.ac.uk

The Problem: Why LLMs Need Help

LLM = Large Language Model

A neural network trained on massive text to generate language (e.g. GPT-4, FLAN-T5). Encoder models like BERT are used for understanding tasks, not generation.

Four key limitations:

1. Knowledge Cutoff
Only knows training data — cannot answer about events after training ended.

2. Hallucination
Generates plausible but **false** content with full confidence.

3. No Private Data Access
Cannot read your company documents, files, or databases — trained only on public web data.

4. Context Window Limits
Can only process a limited number of tokens (word pieces) per input.

Example: You ask "How many days of annual leave do I get?" The LLM answers with generic UK law (28 days) instead of your company's actual policy (30 days + 5 carry-over) — because it was never trained on your internal documents.

Key insight: We need a way to give the LLM access to *your* documents at query time, without retraining it. This is what RAG does — explained on the next slide.

What is RAG?

RAG = Retrieval-Augmented Generation (Lewis et al., 2020)

Combines two capabilities from this module:

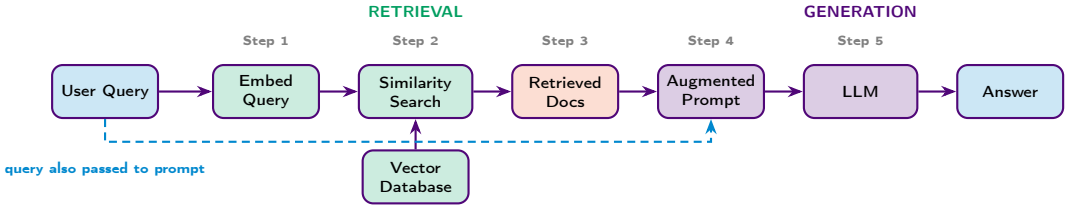
1. **Retrieval** (Weeks 2–5): finding relevant documents via similarity search
2. **Generation** (Week 6): producing text with a transformer model

The RAG Process

1. User asks a question
2. **Retrieve** relevant passages from a knowledge base
3. **Augment** the LLM's input with retrieved context
4. **Generate** an answer grounded in that context

Key insight: We focus on **inference-time RAG**: model weights are unchanged — we augment the input with retrieved documents. (Note: the original Lewis et al. 2020 paper jointly trained retriever and generator end-to-end, but modern practice typically keeps the LLM frozen.)

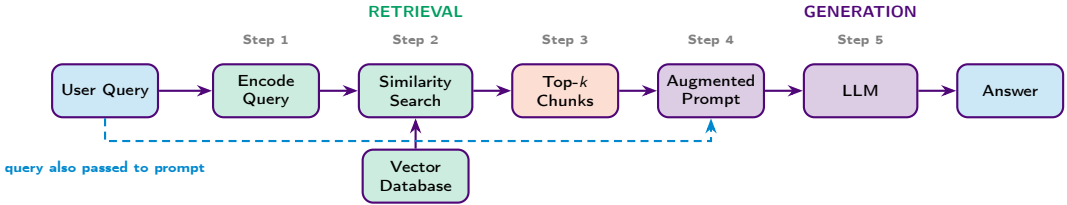
The RAG Architecture



Steps:

1. **Embed** query into vector space
2. **Search** for similar document vectors
3. **Retrieve** top-k documents
4. **Combine** query + retrieved documents into a single prompt
5. **Generate** a grounded answer

The RAG Pipeline: Overview



Two phases:

Indexing (offline, done once):
Load docs → Chunk → Encode → Store in vector DB

Querying (online, per question):
Encode query → Search → Retrieve top-k → Generate

The next slides walk through each step. Code for every step is in the accompanying Colab tutorial.

Step 1: Load Documents

Pipeline Step 1 | *Tutorial Part 3*

Goal: Ingest your knowledge base into the pipeline.

What Happens

Raw documents (PDF, TXT, HTML, Word, etc.) are read into memory as text objects, each with its content and source metadata (e.g. filename, page number).

Key considerations:

- Different file formats need different parsers
- We use **LangChain**, a Python framework for building LLM applications, which provides document loaders for most common formats
- Metadata (filename, page number) is preserved alongside text — this enables **source attribution** in later steps
- At this stage the documents are full-length and unprocessed

Key insight: The quality of your RAG system starts here. If documents are poorly parsed (e.g. garbled PDF extraction), downstream steps cannot recover.

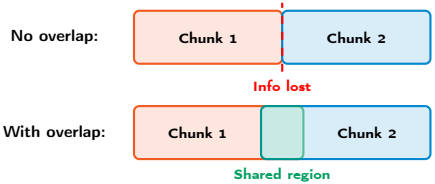
What is Chunking and Why Does It Matter?

Definition

Chunking = splitting long documents into smaller, self-contained segments so each can be independently embedded, indexed, and retrieved.

Why we cannot use whole documents:

1. **Model input limits:** Embedding models truncate beyond their max (e.g. 512 tokens) — text past this is silently lost
2. **Retrieval precision:** Whole documents dilute the embedding with irrelevant text
3. **Context window:** The LLM prompt cannot fit many full documents



Overlap (10–20% of chunk size) duplicates boundary text so split sentences remain complete in at least one chunk.

Defaults: 500–1000 chars, 10–20% overlap. Optimal size is corpus-dependent — evaluate on your data.

Step 2: Chunk Documents

Pipeline Step 2 | *Tutorial Part 5*

Goal: Apply a chunking strategy to split loaded documents into retrieval-ready segments.

Chunks Too Small

Fragments co-referent information. "policy states that..." / "...30 days" — split apart, neither self-contained.

Chunks Too Large

Embedding averages over multiple topics. Leave policy + dress code + parking in one chunk — retrieval returns noise.

Chunking strategies:

Strategy	How It Works	Best For
Recursive (default)	Tries separators in order: paragraph → line → sentence → word	General text
Semantic	Split where similarity between adjacent segments drops	Topic-diverse docs
Structure-based	Split on markup: headers, HTML tags	Manuals, wikis

Goal: Each chunk should be a **self-contained, semantically coherent** unit.

Step 3: Encode Chunks into Embeddings

Pipeline Step 3 | *Tutorial Part 3*

Goal: Convert each text chunk into a numerical vector that captures its meaning.

Why Dense Embeddings, Not TF-IDF?

- **TF-IDF** (Weeks 2–3) matches **exact terms** only — "holiday" and "annual leave" share no words, so TF-IDF gives zero similarity
- **Dense embeddings** capture **meaning** — "holiday" $\approx$ "annual leave" get similar vectors because they appear in similar contexts in training data (the **distributional hypothesis**)
- This extends the idea behind LSI (Week 5), but using non-linear neural networks trained on billions of documents

Sentence Transformers (Reimers & Gurevych, 2019)

Pre-trained transformer models fine-tuned so that semantically similar inputs produce similar vectors. Unlike BERT (which requires feeding sentence *pairs*, giving $O(n^2)$ cost), Sentence Transformers encode each input **independently** — $O(n)$ encoding, then fast cosine similarity comparison.

We use: `all-MiniLM-L6-v2` — 384 dimensions, free, runs locally. **Critical:** Documents and queries **must** use the same model.

What is Chunking?

Definition

Chunking is the process of splitting long documents into smaller segments ("chunks") so each can be independently embedded, indexed, and retrieved.

Why not embed and retrieve whole documents?

- Embedding models have maximum input lengths (e.g. 512 tokens for MiniLM) — text beyond this is truncated
- LLM context windows are limited — you cannot fit many full documents into a single prompt
- Whole documents dilute the relevant passage with irrelevant content, reducing both retrieval precision and generation quality

Key insight: Chunking is one of the most impactful design decisions in RAG. Poor chunking degrades retrieval quality, which cascades into poor answers regardless of LLM capability.

Why Chunking Matters: The Trade-off

Chunks Too Small

- Lose context needed for meaning
- May split co-referent information across chunks
- Increases index size and search cost

Example:
"The annual leave policy states that..."
↓ *(split here)* ↓
"...employees receive 30 days."
Neither chunk is self-contained!

Chunks Too Large

- Relevant content diluted by unrelated text
- May exceed embedding model input limits
- Retrieval returns noisy, low-precision results

Example:
One chunk contains leave policy, dress code, parking rules, and IT security.
The embedding averages over all topics!

Goal: Each chunk should be a **self-contained, semantically coherent** unit — typically one topic, one section, or one policy.

Common Chunking Strategies

Strategy	How It Works	Best For
Fixed size	Split every N characters/tokens	Simple documents, code
Recursive	Try separators in priority order: paragraphs $\rightarrow$ lines $\rightarrow$ sentences $\rightarrow$ words	**General text (recommended default)**
Semantic	Split where embedding similarity between adjacent segments drops	Long documents with topic shifts
Sentence-based	Split on sentence boundaries only	Conversational or short-form content
Structure-based	Split on document markup: headers, sections, HTML tags	Technical docs, manuals, wikis

Practical Recommendations

- Start with **500–1000 characters** per chunk with **10–20% overlap**
- Use **RecursiveCharacterTextSplitter** as your default
- **Evaluate retrieval quality** on your data — optimal chunk size is corpus-dependent

Worked Example: Chunking in Action

Original document (approx. 130 characters):

[frame=single, fontsize=, rulecolor=] Annual leave is 30 days per year. Sick leave requires a doctor's note after 3 days. Remote working is permitted 2 days per week.

Chunked with chunk_size=80, chunk_overlap=20:

Chunk	Content
1	`Annual leave is 30 days per year. Sick leave requires a doctor's`
2	`requires a doctor's note after 3 days. Remote working is permitted`
3	`is permitted 2 days per week.`

Notice: The highlighted text appears in **both** adjacent chunks. This **overlap** ensures that sentences split at a boundary remain complete in at least one chunk.

How RecursiveCharacterTextSplitter Works

The recommended default chunking method. It preserves natural text boundaries by trying separators in priority order:

Preferred — Split on `\n\n` (paragraphs)

↓ if chunk > max size

Split on `\n` (line breaks)

↓ if chunk > max size

Split on `.` (sentences)

↓ if chunk > max size

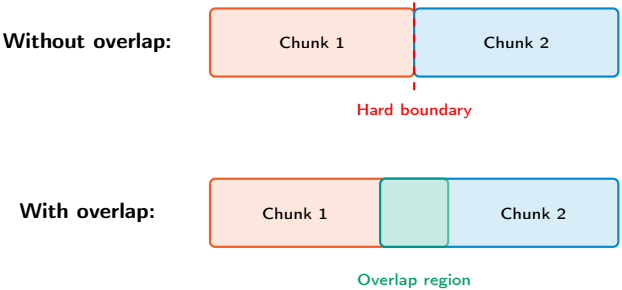
Last resort — Split on (words)

Why "Recursive"?

It **recursively** applies the next separator only to chunks that still exceed the maximum size. Paragraphs are preserved when possible; sentence and word splits are fallbacks. This produces chunks that respect natural document structure.

The Importance of Overlap

Overlap duplicates text at chunk boundaries to prevent information loss.



Concrete example:
Without overlap: "...employees receive" | "30 days annual leave..." (sentence split mid-way)
With overlap: "...employees receive 30 days" | "receive 30 days annual leave..." (complete in both)

Rule of thumb: Set overlap to 10–20% of chunk size. Both are measured in the same unit (characters or tokens, depending on the splitter). E.g. chunk_size=500 chars, overlap=50–100 chars.

Step 4: Store in a Vector Database

Pipeline Step 4 | *Tutorial Part 6*

Goal: Index the embeddings so we can search them quickly at query time.

Why not just compare every vector?
Computing cosine similarity against every stored vector ($O(n)$ per query) works for thousands but becomes too slow for millions. **Vector databases** solve this using **Approximate Nearest Neighbour (ANN)** algorithms — methods that find *approximately* the closest vectors in sub-linear time:

Algorithm	How It Works	Trade-off
HNSW	Traverses a proximity graph to find neighbours	Uses more memory for faster search
IVF	Groups vectors into clusters, only searches nearby ones	May miss vectors in distant clusters
PQ	Compresses vectors to reduce memory usage	Slightly less accurate distances

For this module: We use **Chroma**, a lightweight vector database that runs inside your Python process with no setup needed. Each vector is stored alongside its original text and metadata.

Key insight: Indexing (Steps 1–4) is done **once**. Re-index only when documents change. The cost is amortised across all future queries.

What is a Vector Database?

A vector database is purpose-built for storing, indexing, and querying high-dimensional vectors.

Why not use a regular database?

- Regular databases support **exact match** queries (`WHERE name = 'John'`)
- RAG requires **similarity search** — find the k vectors closest to a query vector
- Brute-force search is $O(n)$ per query — feasible for thousands of vectors but impractical for millions

Vector databases use Approximate Nearest Neighbour (ANN) algorithms to achieve sub-linear query time:

- **HNSW** (Hierarchical Navigable Small World) — proximity graph traversal
- **IVF** (Inverted File Index) — partitions vectors into clusters, searches only nearby clusters
- **PQ** (Product Quantisation) — compresses vectors for memory efficiency, often combined with IVF

Result: Sub-linear query time — search millions of vectors in milliseconds, trading a small amount of recall for large speed gains.

Popular Vector Databases

Database	Type	Best For	Cost
Chroma	Embedded (in-process)	Prototyping, tutorials	Free, open source
FAISS (Meta)	In-memory library	Large-scale research, batch retrieval	Free, open source
Pinecone	Managed cloud service	Production, managed scaling	Paid (free tier)
Weaviate	Self-hosted / cloud	Hybrid search (dense + sparse)	Free / paid
Qdrant	Self-hosted / cloud	High-throughput serving	Free / paid
pgvector	PostgreSQL extension	Adding vector search to existing Postgres	Free

For This Module

We use **Chroma** — it runs in-process with no external server, requires no setup, and is ideal for learning. For production, consider a managed service (Pinecone) or a dedicated engine (Qdrant, Weaviate).

Step 5a: Retrieve Relevant Chunks

Pipeline Step 5a | *Tutorial Part 3–4*

Goal: Given a user query, find the top-k most relevant chunks from the vector database.

Process:

1. Encode the query using the **same** embedding model as indexing
2. Perform ANN search in the vector database
3. Return the k chunks with highest cosine similarity to the query

Choosing k:

- **Too small** (k=1): may miss relevant information → incomplete answer
- **Too large** (k=20): includes irrelevant chunks → noise dilutes context, may exceed LLM input limit
- **Start with** k=3–5 and tune based on evaluation

Worked example:
Query: "How many holiday days do I get?" → top match: "Annual leave allowance is 30 days per year" (similarity: 0.61). Dense embeddings matched "holiday" ≈ "annual leave" — TF-IDF would have missed this.

Step 5b: Generate a Grounded Answer

Pipeline Step 5b | *Tutorial Part 4*

Goal: Combine retrieved chunks + query into a prompt, then generate an answer.

This is not summarisation — it is **knowledge-grounded generation**. The LLM may extract (copy spans from context), synthesise (combine information from multiple chunks), or abstain ("insufficient information").

Prompt Design — Three Essential Elements

1. **Grounding constraint:** "Answer based ONLY on the provided context"
2. **Numbered sources:** [1], [2], [3] — enables attribution and conflict resolution
3. **Fallback instruction:** "If context is insufficient, say so" — prevents fabrication

Why this works: The LLM processes the prompt (context + query) as a single input sequence. Its attention mechanism allows it to focus on the parts of the context that are relevant to the query, effectively "looking up" the answer in the retrieved text rather than guessing from memory.

Without grounding constraints: the LLM may ignore retrieved context and rely on its training data (parametric knowledge), producing unfaithful or hallucinated answers.

Do You Always Need Generation?

No. The retrieval component of RAG is valuable on its own — the LLM is optional.

Retrieval-Only (No LLM)

- Return retrieved passages directly to the user
- User reads and interprets the source text themselves
- No hallucination risk — only original documents shown
- Lower cost, lower latency

Use when: users need exact source text, legal/compliance settings, document search engines

Full RAG (Retrieval + Generation)

- LLM synthesises an answer from retrieved passages
- Handles multi-chunk reasoning and paraphrasing
- Risk of hallucination — requires prompt constraints
- Higher cost (LLM inference)

Use when: users want a direct answer, question answering, customer support

Key insight: The retrieval pipeline (Steps 1–5a) is a complete system on its own — a **semantic search engine**. Adding generation (Step 5b) is an optional layer that trades faithfulness risk for user convenience.

Evaluating RAG: Two Independent Components

Pipeline Step | *Tutorial Part 7*

RAG has two components that can fail independently. Always evaluate both.

Retrieval Evaluation

Did we retrieve the right chunks?

Measures whether the top-k returned chunks are actually relevant to the query.

Key metrics:

- Precision@k, Recall@k
- Mean Reciprocal Rank (MRR)
- Normalised Discounted Cumulative Gain (nDCG)

If this fails: Wrong context → wrong answer, no matter how good the LLM.

Generation Evaluation

Is the answer correct given the context?

Measures whether the LLM produced a good answer from the retrieved chunks.

Key metrics:

- Faithfulness
- Answer Relevance
- Completeness

If this fails: Right context → LLM still produces wrong output.

Key insight: If answers are wrong, diagnose retrieval **first**. Fix chunking and embeddings before modifying the prompt or LLM. The next slides explain each metric with worked examples.

Retrieval Evaluation: Precision and Recall

These metrics measure how well your retrieval step finds the right chunks.

Precision@k — "Of what I retrieved, how much is useful?"

$$\text{Precision@}k = \frac{\text{Number of relevant chunks in top-}k}{k}$$

High precision = the LLM's context window is not wasted on irrelevant text.

Recall@k — "Of everything relevant, how much did I find?"

$$\text{Recall@}k = \frac{\text{Number of relevant chunks in top-}k}{\text{Total relevant chunks in knowledge base}}$$

High recall = all the information needed for a complete answer was retrieved.

The trade-off:

- Increasing k → more chunks retrieved → **recall goes up** (more chance of finding all relevant ones) but **precision may drop** (irrelevant chunks also included)
- Decreasing k → fewer chunks → **precision goes up** (only the best matches) but **recall may drop** (relevant chunks missed)

Retrieval Evaluation: Worked Example

Scenario: Knowledge base has 10 chunks. For a given query, chunks 1, 4, and 7 are relevant (judged by a human). We retrieve top-5, ranked by cosine similarity (highest first).

Rank	Chunk	Relevant?	Content (abbreviated)
1	Chunk 4	✓	"Annual leave allowance is 30 days..."
2	Chunk 9	×	"The office dress code is business casual..."
3	Chunk 1	✓	"Employees may carry over up to 5 days..."
4	Chunk 6	×	"Parking permits must be renewed annually..."
5	Chunk 7	✓	"Part-time staff receive pro-rata leave..."

Calculation:

- Retrieved 5 chunks, 3 are relevant: **Precision@5** $= \frac{3}{5} = 0.60$ (60% of retrieved chunks are useful)
- 3 relevant chunks exist in total, we found all 3: **Recall@5** $= \frac{3}{3} = 1.00$ (100% of relevant info found)

What if k=3? We retrieve only the top 3 (chunks 4, 9, 1) $\rightarrow$ Precision@3 $= \frac{2}{3} = 0.67$, Recall@3 $= \frac{2}{3} = 0.67$.
Precision improved (less noise), but we missed chunk 7 (lower recall). **This is the k trade-off.**

Generation Evaluation: Faithfulness, Relevance, Completeness

Three metrics for evaluating the LLM's generated answer:

Faithfulness — "Is every claim supported by the context?"

Decompose the answer into individual claims. Check each against the retrieved context.
Unfaithful = the model added information not present in any retrieved chunk (**extrinsic hallucination**).

Answer Relevance — "Does the answer address the question?"

The answer may be faithful to the context but fail to actually answer what was asked.
Example: Q: "How many leave days?" A: "Leave must be booked 2 weeks in advance." — faithful but irrelevant.

Completeness — "Does the answer cover all relevant information?"

The answer may be faithful and relevant but miss key details available in the context.
Example: Context mentions 30 days + 5 carry-over. Answer only mentions 30 days — incomplete.

Frameworks like **RAGAS** (Retrieval-Augmented Generation Assessment) automate these by decomposing answers into claims and checking each against context and query.

Generation Evaluation: Worked Example

Context provided to LLM:
[1] "Employees receive 30 days annual leave." [2] "Up to 5 days may be carried over."

Question: "How many holiday days do I get and can I carry them over?"

✓ **Good Answer: "You receive 30 days. Up to 5 days can be carried over."**

Faithfulness: High — both claims are in the context
Relevance: High — directly answers both parts of the question
Completeness: High — covers all relevant context

✗ **Bad Answer: "You receive 30 days, plus bank holidays. Carry-over is unlimited."**

Faithfulness: Low — "bank holidays" and "unlimited" are fabricated (extrinsic hallucination)
Relevance: High — it does address the question
Completeness: Misleading — includes false information alongside true

Key insight: An answer can be relevant but unfaithful (addresses the question but makes things up), or faithful but irrelevant (true but doesn't answer the question). You need all three metrics.

Common RAG Failure Modes

Pipeline Step | *Tutorial Part 8*

Failure Mode	Symptom	Solution
Poor retrieval	Irrelevant chunks returned	Improve embeddings, chunking, or add reranking
Lost in the middle	LLM ignores mid-context info	Place most relevant chunk first or last
Extrinsic hallucination	Claims not in context	Strengthen prompt constraints
Incomplete answer	Key info missing	Increase k; improve chunk overlap
Context overflow	Too much text for LLM	Reduce k; compress chunks
Mismatched models	Incompatible embeddings	Same model for docs and queries

The "Lost in the Middle" Problem (Liu et al., 2023)

LLMs exhibit a U-shaped attention pattern over long contexts: they attend most to the *beginning* and *end*, with degraded performance on middle positions. Reorder chunks so the most relevant appears first.

RAG vs Fine-Tuning

RAG

Non-parametric knowledge at inference time

- Knowledge is external, updateable
- Model weights unchanged
- Supports source attribution
- No GPU training cost

Use for: dynamic facts, private docs, traceability

Fine-Tuning

Update parametric knowledge via training

- Knowledge encoded in weights
- Fixed until retrained
- No retrieval latency
- Adapts behaviour and reasoning

Use for: domain style, specialised reasoning, jargon

Key insight: Production systems often combine both: fine-tune for domain behaviour and reasoning patterns, use RAG for factual grounding and source attribution.

Advanced RAG Techniques

When basic RAG is insufficient — add complexity guided by evaluation:

Technique	What It Does
Hybrid search	Combines dense (semantic) + sparse (lexical) via Reciprocal Rank Fusion
Reranking	Cross-encoder rescores initial candidates ($O(k)$ joint encoding vs $O(n)$ bi-encoder)
Query expansion	Multiple query reformulations to improve recall
HyDE (Gao et al., 2023)	LLM generates hypothetical answer; its embedding becomes the query
Parent document retrieval	Index small chunks, return larger parent context to the LLM
Multi-hop RAG	Iterative retrieve-and-reason across multiple documents

Key insight: Start with basic RAG. Add complexity only when evaluation metrics identify a specific bottleneck (e.g. low retrieval recall $\rightarrow$ try hybrid search; low precision $\rightarrow$ try reranking).

Summary: The RAG Pipeline

	Step	What Happens	Tool (Tutorial)
1	Load documents	Parse files into text + metadata	LangChain loaders
2	Chunk	Split into self-contained segments	RecursiveCharacterTextSplitter
3	Encode	Convert chunks to dense vectors	all-MiniLM-L6-v2
4	Store	Index vectors for fast ANN search	Chroma
5a	Retrieve	Find top-k chunks for query	Cosine similarity + ANN
5b	Generate (optional)	Grounded answer from context	FLAN-T5

Key takeaways:

- RAG augments the LLM's input with retrieved documents — no model retraining needed
- Dense embeddings enable semantic retrieval (meaning-based, not keyword-based)
- Chunking and prompt design are the highest-impact design decisions
- Generation is optional — Steps 1–5a alone form a semantic search engine
- Always evaluate retrieval and generation **separately**

Hands-on: The Colab tutorial implements every step with code. Available on the module VLE.

References

Key Papers:

- Lewis et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — `arxiv.org/abs/2005.11401`
- Reimers & Gurevych (2019), "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" — `arxiv.org/abs/1908.10084`
- Liu et al. (2023), "Lost in the Middle: How Language Models Use Long Contexts"
- Gao et al. (2023), "Precise Zero-Shot Dense Retrieval without Relevance Labels" (HyDE) — `arxiv.org/abs/2212.10496`

Tools: LangChain (`python.langchain.com`), Chroma (`trychroma.com`), Sentence-Transformers (`sbert.net`), RAGAS (`docs.ragas.io`)

Module connections: Week 4 (Similarity), Week 5 (LSI / Evaluation), Week 6 (Transformers)

Questions?

Prof. Georgina Cosma

g.cosma@lboro.ac.uk

Office hours: By appointment